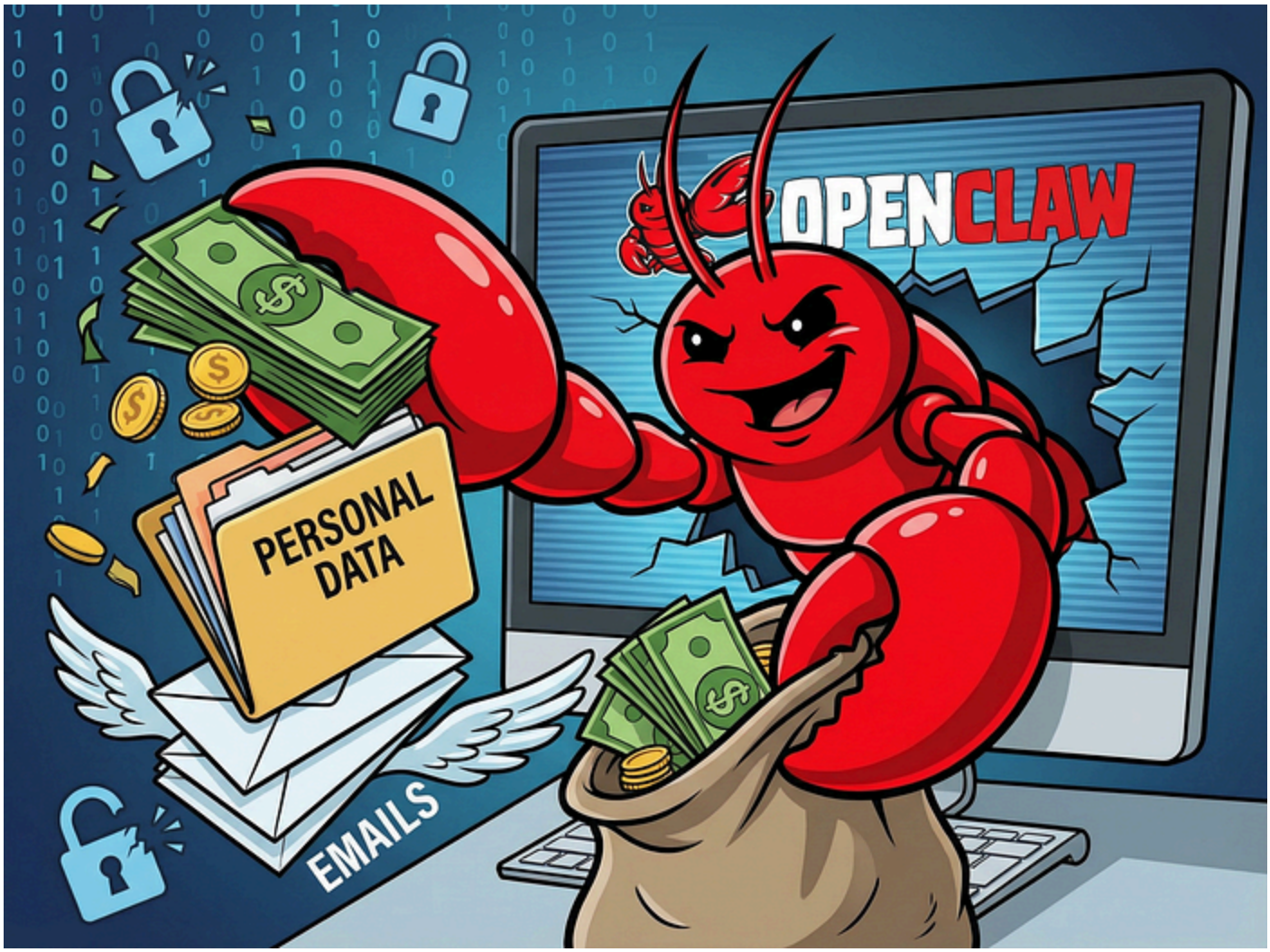
Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



Think Your Emails Are Safe? OpenClaw (Clawdbot) Proves Otherwise

Before you plan delegating real work to OpenClaw, you have to understand exactly why it's vulnerable and how you can mitigate the risks.

Agent Native Follow

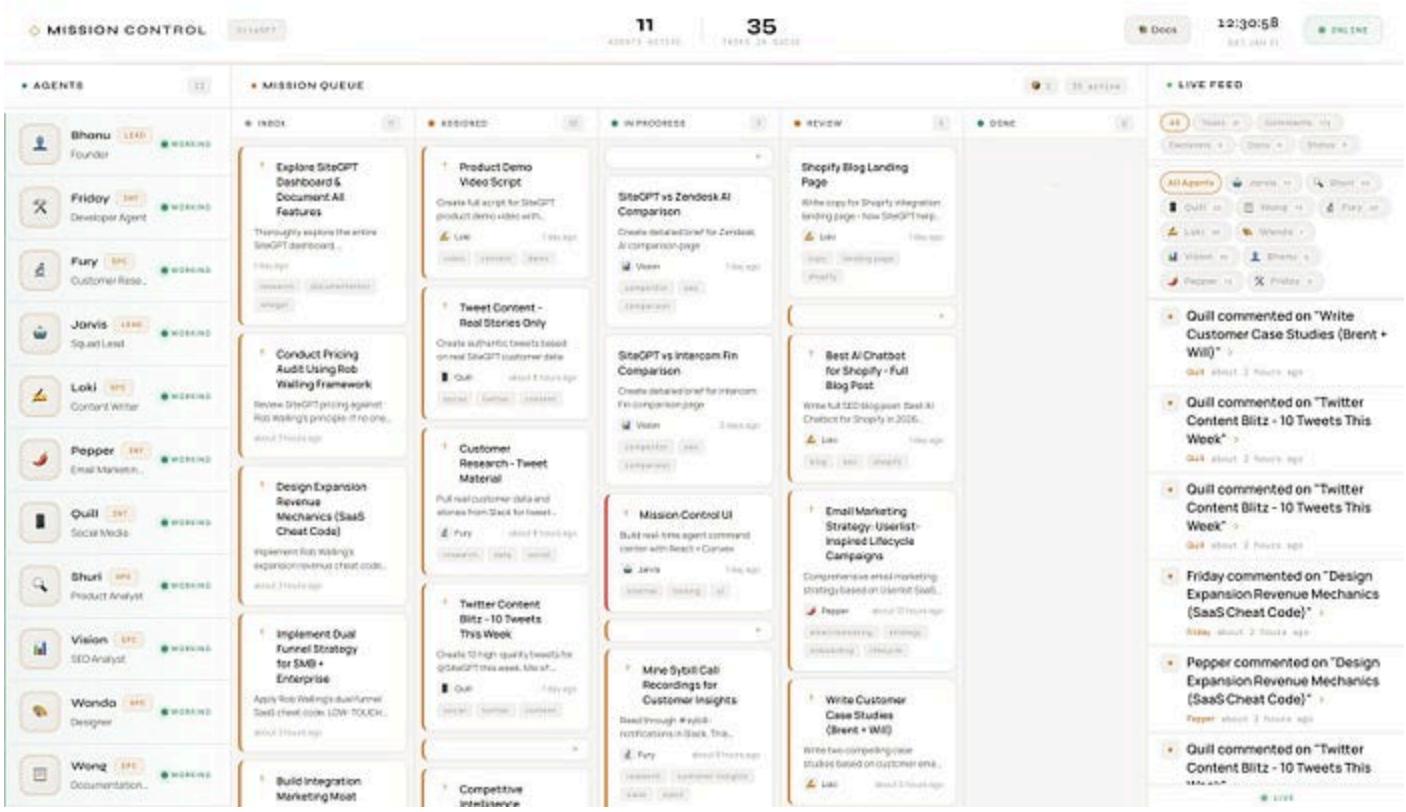
~10 min read · February 1, 2026 (Updated: February 1, 2026) · Free: No

Shodan scan in late January found more than 1800+ instances of OpenClaw on the public internet, over half were running with an unauthenticated gateway and another quarter trusted spoofed `X-Forwarded-For` headers to decide if a request came from `127.0.0.1`.

The uncomfortable truth is that thousands of well-intentioned builders gave a fresh, fast-moving tool privileged access to their digital lives before they had a threat model.

At the same time, OpenClaw's adoption shows why developers are willing to take that bet.

Bhanu Teja's "Mission Control" demonstrates what happens when you treat agents like a team, e.g. multiple specialized sessions, shared workspace, persistent memory, and cron-based wakeups orchestrated like real coworkers.



https://x.com/pbteja1998/status/2017662163540971756

Elvis Sun's build-in-public story shows the solo-founder version where agents triaging user feedback, shipping fixes, automating deployments, even spinning up admin tooling, the "system that builds the product" vibe.

AFTER: Automated Pipeline

WHY THIS MATTERS

BEFORE	AFTER
Agents ship → wait for Elvis	Agents ship → CI validates → merge
Elvis forgets → prod breaks	Order enforced: db first, then app
1 human = bottleneck	Humans review, machines execute
"Hey Elvis, can you push db?"	(silence, it just works)

THE META POINT

```
Agents built this pipeline so more agents can ship faster.

We're not building the product anymore.
We're building the system that builds the product.
```

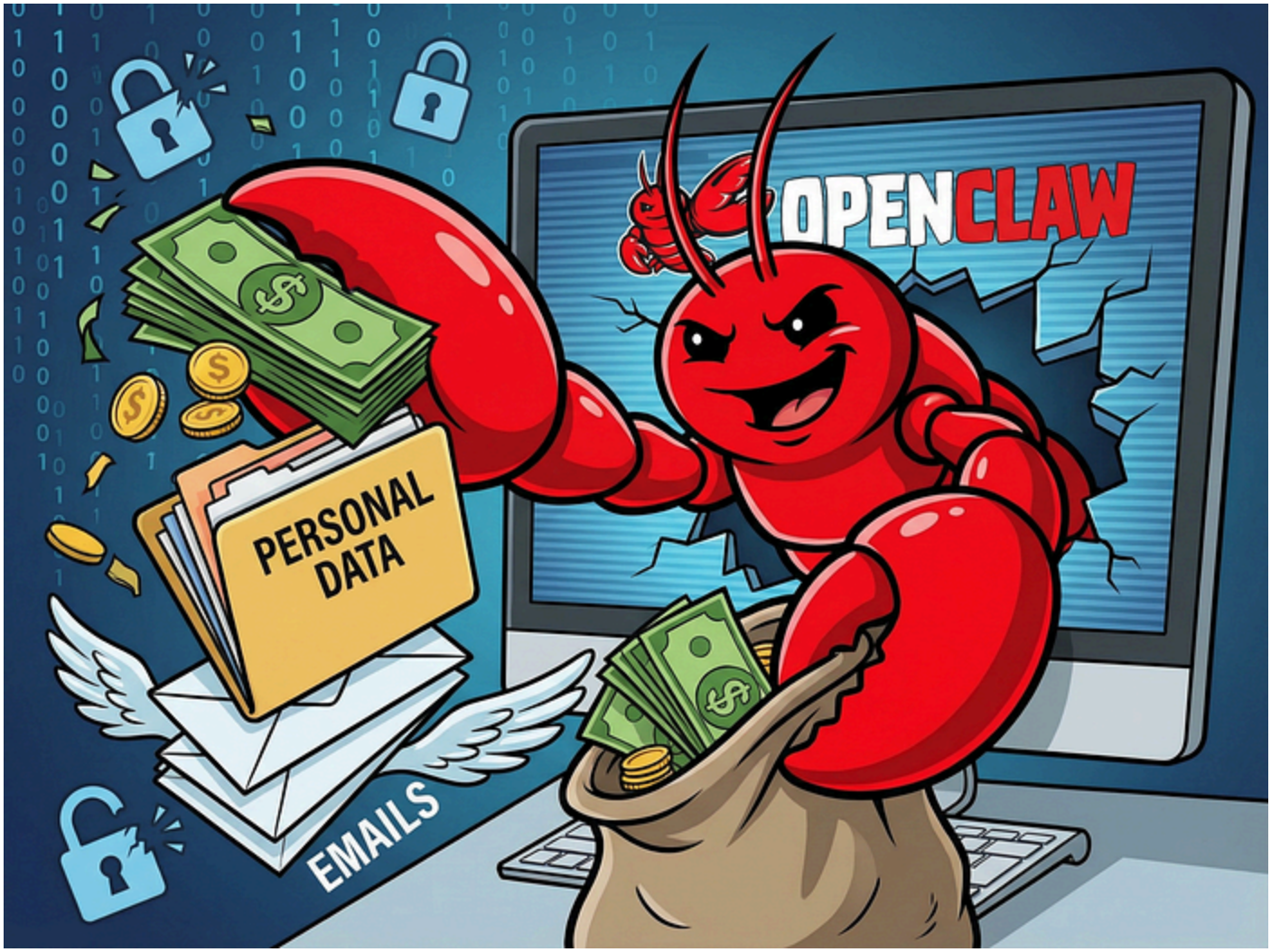
https://x.com/elvissun/status/2017783446664548551

So yes, OpenClaw is a capability breakthrough but it's also a security footgun if you run it like a weekend side project and then quietly connect it to the keys to your life (or your company laptop).

In this guide, I'm going to turn the chaos into a practical reference by explaining how OpenClaw works under the hood, what the real threat model looks like (prompt injection included), the most common deployment mistakes, a hardened setup you can copy, and how to scale from one agent to a coordinated multi-agent crew.

> *If you haven't received it yet, Issue #1 is live for agentic SaaS patterns winning in 2026, enjoy the issue.*

Let's dive into OpenClaw.



Understanding OpenClaw's architecture

OpenClaw instance runs as a **gateway daemon** on a server.

The gateway listens on a WebSocket/HTTP port and routes messages between the AI model and external channels (Telegram, Slack, email, etc.).

It manages **sessions**, which are persistent conversations with unique session keys where each session has its own conversation history, memory files and tool access.

Messages arrive through the gateway, which loads the session's history from disk, invokes a model (Anthropic Claude, OpenAI, Gemini, etc.), passes the result back to the user and appends the interaction to a JSONL history file.

Sessions can be interactive or **isolated** (spawned for cron jobs and terminated immediately).

OpenClaw exposes a tool API that gives the agent "hands" such as:

- **File-system skill**: read and write files within a workspace directory (not the whole disk).
- **Shell skill**: run terminal commands on the host.
- **Browser skill**: drive a headless Chromium instance to scrape websites or interact with UIs.
- **Custom skills**: load arbitrary Python or JavaScript modules from a `skills/` folder.

These tools make the agent powerful, but also mean that any prompt injection or misconfiguration can result in remote code execution.

OpenClaw can store API keys and OAuth tokens in plain `.env` files and a SQLite database, so a single command like `cat ~/.openclaw/.env` can leak all secrets.

Session keys and multiple agents

OpenClaw sessions are addressed by *session keys* (`agent:role:instance`).

Creating multiple sessions with different keys gives you multiple agents with independent contexts.

Bhanu Teja's Mission Control uses a **team of ten agents**, each with a dedicated role and session key (e.g., `agent:developer:main` for a developer agent named "Friday").

```plaintext
agent:main:main              → Jarvis (Squad Lead)
agent:product-analyst:main   → Shuri
agent:customer-researcher:main → Fury
agent:seo-analyst:main       → Vision
agent:content-writer:main    → Loki
agent:social-media-manager:main → Quill
agent:designer:main          → Wanda
agent:email-marketing:main   → Pepper
agent:developer:main         → Friday
agent:notion-agent:main      → Wong
```

Each session reads a `SOUL.md` file defining its personality (e.g., Shuri the product analyst is skeptical and hunts edge cases) and a shared `AGENTS.md` file describing available tools and protocols.

This design allows specialization without cross-contamination of memory.

Cron jobs and heartbeats

Agents don't stay awake all the time.

OpenClaw has an internal cron scheduler that can trigger messages to any session on a schedule.

Mission Control configures **heartbeats** every fifteen minutes where each agent wakes up, loads its `WORKING.md` file (which tracks the current task and next steps), checks for @mentions or assigned tasks, performs work if necessary and then sleeps.

Staggered heartbeats (Pepper at 00 minutes, Shuri at 02 minutes, etc.) avoid thundering herds and reduce API costs.

Cron-based wake-ups are the backbone of asynchronous, multi-agent coordination.

The workspace and memory files

OpenClaw persists state in a workspace directory. Key files include:

- **`SOUL.md`**: defines the agent's identity—name, role, personality traits and what it cares about.
- **`AGENTS.md`**: explains how agents should behave, where to store memory, how to use tools and how to interact with Mission Control.
- **`memory/WORKING.md`**: the agent's working memory; it records the current task, status and next steps. Agents read this on every heartbeat to remember what they were doing.
- **Daily notes (`memory/YYYY-MM-DD.md`)**: a running log of actions taken on a given day.
- **`memory/MEMORY.md`**: long-term memory of important facts or lessons learned.

The multi-agent orchestration layer

Mission Control adds a **shared database** (Convex) and user interface on top of OpenClaw.

Tasks move through states (`inbox → assigned → in progress → review → done → blocked`).

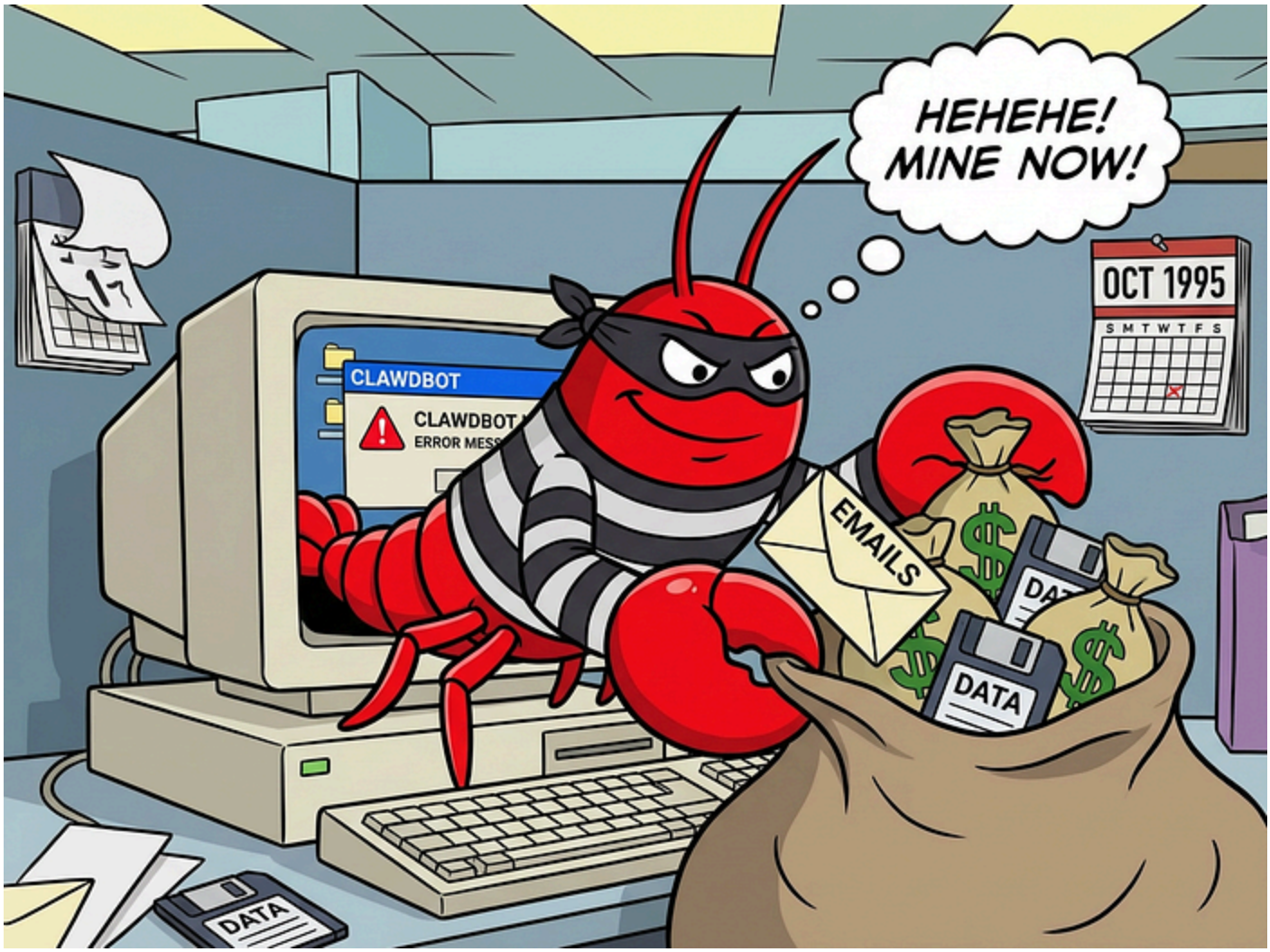
Agents post comments, attach documents and @mention each other. A notification daemon polls the database and forwards mentions to sleeping agents.

A daily stand-up cron summarises completed work, tasks in progress, blockers and decisions, sending a digest to the human owner.

This infrastructure lets a human supervise many agents without micromanaging each one.

> If you are building with OpenClaw, please share your experience in the comments.

Given all these capabilities, it's very important to understand the security.



The security landscape: why caution is essential

If you run a [penetration test against an out-of-the-box OpenClaw agent](), you can achieve a *91 % prompt-injection success rate* and extracted internal system prompts, tool configurations and memory files in the very first conversation turn.

This shows that naive agents will happily leak secrets or follow malicious instructions if you don't take precautions.

The problem isn't unique to OpenClaw. Models like Claude, Gemini and Codex all misbehaved in similar ways but OpenClaw's extensive tool access amplifies the damage.

At the infrastructure level, misconfiguration magnified the threat.

Shodan scan found **1,842** publicly accessible OpenClaw instances. Over **62%** had no authentication, and another **28%** could be taken over by forging `X-Forwarded-For` headers because the software trusted any request appearing to originate from `127.0.0.1`.

Apparently, you can also exploit the agent's ability to run shell commands to read `.env` files containing API tokens. You can use the browser skill to exfiltrate cookies or session data.

This means that you can connect directly to email, files, messaging platforms and system tools without having IAM or secrets controls.

This begs the question: what happens when people start using this tool on company devices? A shadow-IT nightmare is coming.

Beyond open ports, **prompt injection** is the scariest threat.

Hackers can embed malicious instructions in emails, calendar invites or web pages.

When the agent processes this content, the injection overrides its safety instructions like a poisoned calendar invite can cause the agent to execute `rm -rf /`.

Because the model is designed to follow human-like instructions, it will treat the injected text as user intent if the injection occupies enough attention in the context window.

This risk exists even if your network is locked down, the attack vector arrives through normal channels like email or websites.

Why the tool leaks by default

OpenClaw prioritised accessibility and flexibility over enterprise-grade security.

Default deployments trust local traffic and require manual configuration to enable authentication.

The explosive growth of the user community means many people run the software without hardening it, then grant it access to real e-mail accounts, calendars and password vaults.

Threat models are changing in a multi-agent world.

The attack surface becomes the *model's prompt*, the agent's memory files and any external data it consumes. The security problem is about preventing the model from being socially engineered.

Simon Willison calls this the "lethal trifecta" of AI agents, they have access to private data, they process untrusted content and they can communicate externally.

If all three are present (and they are in OpenClaw) then attackers only need to embed a clever sentence in a webpage to hijack your agent.

Secure deployment workflow

The good news is that you don't have to abandon OpenClaw.

You just need to treat it like a privileged service account and apply a zero-trust mindset.

Sandboxing: contain the blast radius

1. **Run the agent in a virtual machine (VM)**. Use UTM or VMware Fusion to spin up a clean macOS or Linux VM. Install OpenClaw *inside* the VM, not on your host. This isolates the agent's file system and prevents it from reading your personal directories. If something goes wrong, you can delete the VM.
2. **Avoid privileged containers**. Don't run the gateway as `root` or mount your entire home directory into the container. You can use gVisor or Kata to intercept system calls. If you use Docker, set `security_opt: no-new-privileges:true` and drop all capabilities in `docker-compose.yml`.

3. **Disable localhost trust**. In your `.env` or configuration file, set `LOCALHOST_TRUST_ENABLED=false`. Configure your reverse proxy (Nginx, Caddy, Traefik) to set proper `X-Forwarded-For` headers and require authentication on port 18789. Many of the Shodan exposures were caused by misconfigured proxies.
4. **Enable authentication**. Recent versions of OpenClaw support OIDC authentication. Configure an OAuth provider (Auth0, Google, etc.) and require login for every request. Resist the temptation to expose the WebSocket without auth because "it's just me using it."

Strengthen the model against prompt injection

1. **Use an inoculation prompt**. Install an Advanced Cognitive Inoculation Prompt (ACIP). It primes the model to detect and ignore instructions that deviate from the user's intent.
2. **Sanitize untrusted inputs**. Treat any data from emails, calendar invites or web pages as untrusted. Strip HTML, hide hidden text and remove suspicious patterns before feeding it into the model. You can also build a middleware that "taints" external data and prohibits it from being executed as commands.
3. **Implement a human-in-the-loop (HITL)**. For high-risk actions (deleting files, sending money, editing production databases), require explicit human approval. You can allow read-only by default and using UI confirmations for write operations. OpenClaw's skill system can be patched to enforce a `confirm()` call before executing dangerous commands.
4. **Monitor context windows**. Attackers can overwhelm the model's working memory by including long malicious payloads. Limit the amount of untrusted content the agent loads at once. Keep the conversation memory small and rely on external storage for long-term information.

Isolate credentials and secrets

1. **Give your agent its own accounts**. Create a separate Gmail or Workspace account for the bot. Use it for email, calendar and drive access. Do not share your personal inbox. Only share specific calendars or documents with the bot's account using read-only permissions.
2. **Use a dedicated password vault**. OpenClaw stores secrets in plain files by default. Instead, create a separate vault in a password manager like 1Password called "Shared with OpenClaw." Generate a service account with access *only* to that vault. Store all API keys there and teach the agent to retrieve secrets using the `op` CLI, not by reading files. Never write secrets to memory files or logs.
3. **Separate API quotas**. For services like Slack, Telegram or AWS, create new API keys for the bot. Monitor usage. If the bot behaves unexpectedly, you can revoke its key without affecting your personal accounts.

Configure email and calendar access carefully

1. **Install a Gmail CLI**. Tools like `gog` allow the agent to access its own mailbox. Instruct the bot to check email every few minutes via a cron job and to follow a strict protocol: it may act on messages from trusted senders (e.g., your addresses), but must treat all other senders as read-only.
2. **Share calendars read-only**. Log into your personal Google account and share the calendars you want the agent to access with the bot's account. Accept the invitation from within the VM. Start with read-only access; upgrade only if necessary.
3. **Teach scheduling preferences**. Create a `SCHEDULING.md` file containing your working hours, preferred meeting times, VIP exceptions, buffers, and special rules (e.g., avoid scheduling after 3 p.m. on

Fridays). Reference this document so the agent consults it when proposing times. Agents like "Jarvis" in Mission Control have become skilled executive assistants by following these preferences.

4. **Apply updates promptly**. OpenClaw is under active development, and community members (including security researchers) regularly submit patches. Maintainers fix vulnerabilities in hours or days. Subscribe to the project's release feed and update your deployment regularly.

I will write more about OpenClaw in coming days, stay safe and keep building!

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

Kimi K2.5 + Agent Swarms Beat US AI Labs: Open Source vs OpenAI, Anthropic & Google

If you missed this week's Kimi K2.5 announcement, you have to read on.

medium.com

GLM-4.7-Flash on 24GB GPU (llama.ccp, vLLM, SGLang, Transformers)

GLM-4.7-Flash is one of those rare open-weights releases that changes what "local-first" can realistically mean for...

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com
#openclaw#clawdbot#ai-security#ai-agent-security#prompt-injection-attack

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel